

```

import numpy as np

# Task 1: Mean Squared Error (MSE)

def calculate_mse(predictions, targets):
    predictions = np.array(predictions)
    targets = np.array(targets)

    mse = np.mean((predictions - targets) ** 2)
    return mse

predictions = [2.5, 0.0, 2.1, 8.0]
targets = [3.0, -0.5, 2.0, 7.0]

print("===== Task 1 =====")
print("Mean Squared Error:", calculate_mse(predictions, targets))

# Task 2: Mean Absolute Error (MAE)

actual_ratings = np.array([4, 3, 5, 2, 1])
predicted_ratings = np.array([5, 2, 4, 2, 1])

mae = np.mean(np.abs(actual_ratings - predicted_ratings))

print("\n===== Task 2 =====")
print("Mean Absolute Error:", mae)

# Task 3: Binary Cross-Entropy (BCE)

def binary_cross_entropy(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    # Avoid log(0)
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)

    loss = -np.mean(
        y_true * np.log(y_pred) +
        (1 - y_true) * np.log(1 - y_pred)
    )

    return loss

```

```

y_true = [1, 0, 1]
y_pred = [0.9, 0.2, 0.7]

print("\n===== Task 3 =====")
print("Binary Cross Entropy:", binary_cross_entropy(y_true, y_pred))

# Task 4: Categorical Cross Entropy

y_true = np.array([0, 0, 1])
y_pred = np.array([0.2, 0.3, 0.5])

epsilon = 1e-15
y_pred = np.clip(y_pred, epsilon, 1)

cce = -np.sum(y_true * np.log(y_pred))

print("\n===== Task 4 =====")
print("Categorical Cross Entropy:", cce)

# Task 5: Real World Example

print("\n===== Task 5 =====")

print("App: Zomato")
print("Scenario: Predict whether a user will order food or not.")
print("Loss Function: Binary Cross-Entropy")
print("Reason: The prediction has only two possible outcomes:")
print("Order (1) or Not Order (0). BCE is designed for binary classification.")

```

Explanation

Task 1: Mean Squared Error (MSE)

Formula

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- y_i = Actual value
- $\hat{y}_i$ = Predicted value

Example:

Actual Prediction Error Squared Error

3	2.5	0.5	0.25
-0.5	0	0.5	0.25
2	2.1	-0.1	0.01
7	8	-1	1.00

Average of squared errors = **MSE**.

Why use MSE?

- Penalizes large errors more heavily because errors are squared.
 - Commonly used in regression problems.
-

Task 2: Mean Absolute Error (MAE)

Formula

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Given:

Actual Ratings:

[4,3,5,2,1]

Predicted Ratings:

[5,2,4,2,1]

Absolute errors:

1

1

1

0

0

Average:

MAE = 0.6

Why use MAE?

- Easy to interpret because it measures the average absolute error.

- Less sensitive to outliers than MSE.
-

Task 3: Binary Cross-Entropy (BCE)

Formula

$$BCE = -\frac{1}{n} \sum [y \log(p) + (1 - y) \log(1 - p)]$$

where:

- y = True label (0 or 1)
- p = Predicted probability

Example:

True Labels:

[1,0,1]

Predictions:

[0.9,0.2,0.7]

The BCE loss is low because the predicted probabilities are close to the correct labels.

Why clip predictions?

epsilon = 1e-15

This avoids taking $\log(0)$, which is undefined.

Task 4: Categorical Cross-Entropy (CCE)

Formula

$$CCE = -\sum y_i \log(p_i)$$

Given:

True Label:

[0,0,1]

Predicted probabilities:

[0.2,0.3,0.5]

Only the true class contributes to the loss:

Loss = $-\log(0.5)$

The higher the predicted probability for the correct class, the lower the loss.

Why use CCE?

- Designed for multi-class classification.
 - Works together with the Softmax activation function.
-

Task 5: Real-World Scenario

App: Zomato

Scenario: Predict whether a user will place an order after viewing a restaurant.

Problem Type: Binary classification

Best Loss Function: Binary Cross-Entropy (BCE)